



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

VPC Laboratorios: Gender Recognition, Car Model identification & Attention Visualization

TRABAJO TECNOLOGÍAS DEL LENGUAJE HUMANO

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Gómez Corral, Miquel

CAPÍTULO 1

Introducción

En esta memoria se presentan los trabajos seleccionados para la asignatura de Visión por Computador: Gender Recognition, Car Model Identification y Attention Visualization.

Se presentan los pasos que se han seguido para llegar a ellos y los resultados obtenidos en cada uno de los ejercicios.

Hardware utilizado

Los experimentos se han realizado en mi máquina personal: GPU NVIDIA RTX 5070 (12 GB VRAM), procesador AMD Ryzen 9 9000X, 64 GB de RAM y Ubuntu 24.04 LTS.

Todos los modelos han podido ser entrenados y evaluados sin problemas.

1.1 Código entregado

Se han entregado 3 notebooks de Jupyter, cada uno correspondiente a un ejercicio. Estos se encuentra en la carpeta /notebooks junto al resto del repositorio con el código necesario para ejecutar cada uno de los ejercicios.

- `gender_prediction.ipynb`: Ejercicio 1 predicción de género.
- `car_model_identification.ipynb`: Ejercicio 2 identificación de modelos de coches.
- `attention_visualization.ipynb`: Ejercicio 3 visualización de atención.

El código asociado a esto ejercicios se puede encontrar en /app repartidos el sus correspondientes subcarpetas. El punto de acceso a estos es el archivo `main.py` que se encarga de cargar los modelos y ejecutar los experimentos necesarios para cada ejercicio.

Está auto documentado y se puede usar siguiendo las instrucciones del README para instalar las dependencias y ejecutar cada uno de los ejercicios.

CAPÍTULO 2

Ejercicios

2.1 Ejercicio 1: Gender Recognition

Para realizar esta tarea, se ha usado como referencia el paper adjunto en el enunciado, ha sido de gran ayuda.

Ya que la tarea se dividía en dos objetivos, se ha intentado conseguir primero el score de $\geq 98\%$ accuracy en validación, para luego reducir el tamaño del modelo y conseguir el objetivo de 95 % accuracy con menos de 100k parámetros.

2.1.1. Datos

Los datos provistos venían en un formato muy cómodo de numpy y ya divididos en entrenamiento y test (test = validación). Se han cargado y se han convertido en un dataset de PyTorch para facilitar su uso en el entrenamiento de los modelos y poder aplicar técnicas de data augmentation.

2.1.2. Métodos probados

En la tabla 2.2 se aprecia un resumen de los resultados obtenidos con cada técnica probada. Parte por parte se desglosa en el siguiente apartado.

Con una pipeline de entrenamiento estándar, se han probado diferentes arquitecturas de CNN, con diferentes hiperparámetros y técnicas de regularización.

Siguiendo el paper de referencia y sus nombres, se ha iniciado con una CNN del formato 'T', con `torch.optim.SGD`, `torch.nn.CrossEntropyLoss` y sin técnicas de regularización. Se ha modificado ligeramente para quitar la capa de `AdaptiveAvgPool2d` a (8,8) que añadían en el paper. Esta causaba que el entrenamiento fuera inestable y dependía de factores como la semilla para entrenar bien.

Esto nos dejaba con valores 'bajos' de accuracy, de un 93 %, por lo que se han ido probando las siguientes técnicas de regularización:

- Usar AdamW en vez de SGD.
- Usar label smoothing con $\epsilon = 0,1$.
- Usar data augmentation con técnicas de RandomRotation, RandomHorizontalFlip, RandomGrayscale, RandomResizedCrop, RandomErasing, GaussianNoise y ColorJitter

La mayoría de estas han terminado mejorando el accuracy, aun que algunas como el ColorJitter o el RandomResizedCrop solo ralentizaban el entrenamiento, sin llegar a dar una mejora al modelo.

Con esto, se ha llegado a superar el 96,0% – 96,5% de accuracy en validación, pero aún estaba lejos del objetivo de 98%. En el paper, se usan varias versiones de la misma CNN pero entrenadas con seeds y usadas como un ensemble. Sin embargo, se ha querido implementar una mejora más al proceso antes de llegar a esto.



Figura 2.1: Muestras fallidas por el modelo antes de aplicar ensemble

Si nos fijamos, a parte de no saber si Michael Jackson es hombre o mujer, el modelo también tiene problemas con personas con gafas, gorras y en general con aquellas a las que no se les ve el pelo (no todos son así claro). Esto nos lleva a pensar que depende mucho de estas características para hacer la predicción, y que si no las tiene, el modelo se confunde.

Por eso, se ha implementado un **aumento de datos específico** que, a partir del concepto del RandomErasing, pretende centrarse en zonas específicas de la imagen, en concreto en la zona del pelo.

Con este aumento, se consigue pasar de un 96.5 % a un 97.1 % de accuracy en validación. Aún así, no se ha conseguido llegar al 98 % de accuracy, por lo que ha decidido probar a ajustar el umbral de predicción y hacer inferencia usando cada imagen y su versión invertida horizontalmente. Con esto alcanzamos un 97.3 % de accuracy, solo usando este modelo.

Luego, siguiendo las indicaciones del paper, se ha implementado un ensemble de 3 modelos con diferentes seeds, AdamW, label smoothing y los data augmentation que han demostrado funcionar incluyendo la personalizada, subiendo un pelín el accuracy a 97.5 %.

Al ver que se llegaba a un tope, se ha cambiado directamente la arquitectura del modelo. La vista el paper llega a un tope con el seput actual, por lo que se ha usado una Resnet con 3 skipconnections y dos dos convoluciones por bloque.

Con esta nueva arquitectura, se ha conseguido llegar a un 98.4 % de accuracy en validación, y con el ensemble de 3 modelos, se ha llegado a un 98.6 %, superando el objetivo marcado. Tras algunas pruebas se ha superado esta marca añadiendo otra capa de convolución a cada bloque, llegando a un 98.6 % con un solo modelo y a un 98.8 % con un ensemble, siendo este el mejor resultado obtenido.

2.1.3. Modelo pequeño

Para conseguir el objetivo de un modelo con menos de 100k parámetros y un accuracy de al menos 95 %, se ha optado por usar exactamente la misma arquitectura que el modelo grande, pero esta vez usado un `AdaptiveAvgPool2d((6, 6))` como entrada a la parte Densa del modelo, 61 canales en vez de 64 en la última capa convolucional, lo que a su vez reduce el número de neuronas de la primera capa densa.

Esto hace que el número de parámetros se reduzca a 99.0K, y con la buena pipeline de entrenamiento, se ha conseguido fácilmente un accuracy en validación se mantenga por encima del 95 %, llegando a un 96.8 %.

Modelo	Técnica	Accuracy %
Modelo I	Base sin <code>AdaptiveAvgPool2d</code>	93.0
	Mejores técnicas	96.5
	Aumento específico	97.1
	Horizontal flipping	97.3
	Todo + Ensemble	97.6
ResNet 1	Todo	98.2
	Todo + Ensemble	98.6
ResNet 2	Todo	98.6
	Todo + Ensemble	98.8
Modelo I Pequeño	Todo	96.8
	Todo + Ensemble	97.0

Tabla 2.1: Resultados de accuracy para diferentes técnicas aplicadas.

Como vemos, el tamaño del modelo no ha tenido tanta importancia a la hora de conseguir un buen accuracy y ha sido más la configuración del entrenamiento, los data augmentation y el buen uso del modelo en inferencia.

2.2 Ejercicio 2: Car Model Identification

Para esta tarea, partimos usando el código del ejercicio anterior, toda la pipeline de entrenamiento y data augmentation se ha mantenido, aunque se han ajustado algunos hiperparámetros como el learning rate y se ha eliminado el aumento de datos personalizado.

Ahora la diferencia será que no solo usamos una CNN, sino dos y ya entrenadas. Usaremos una ResNet-50 y una VGG-16 preentrenadas en ImageNet tanto para crear una red bilinear con ambas como para solo usar una y crear redes siamesas.

2.2.1. Datos

De forma similar al ejercicio anterior, los datos venían en formato numpy y ya divididos en entrenamiento y test (validación). Se han cargado y convertido a un dataset de PyTorch para facilitar su uso en el entrenamiento de los modelos.

Ahora, estas imágenes tienen un tamaño de 250x250. Por los tamaños de las salidas de las CNNs usadas, se han reescalado a 224x224 para que su último mapa convolucional devuelva una salida de 7x7 (ambas), que es lo que se necesita para la parte bilinear del modelo.

2.2.2. Entrenamiento

Como ya sabemos, para entrenar una red bilinear usando dos CNNs ya entrenadas, únicamente tenemos que combinar las salidas de ambas CNNs, hacer un producto externo entre todos los canales de las salidas. Este tensor se proyecta a un vector y se normaliza según el tamaño del mapa convolucional, que en este caso es de 7x7, y se le aplica una raíz cuadrada seguida de una normalización L2.

Finalmente, este vector tendrá un tamaño equivalente a multiplicar los canales de ambas redes (por ejemplo, $2048 \times 512 = 1048576$ características si cruzamos la ResNet y la VGG), y se pasa a una sola capa lineal que da 20 salidas, una por cada clase.

Para hacer efectivo el entrenamiento de esta red bilinear, se congelan las dos redes convolucionales, y se entrena solo la capa lineal final hasta que se estabilice. Luego, se reduce el learning rate en dos órdenes de magnitud y se descongelan las CNNs para hacer un fine-tuning de toda la red.

2.2.3. Técnicas probadas

Con el mismo proceso de entrenamiento que el ejercicio anterior, no se ha tenido que cambiar apenas nada y se han podido probar los tres posibles enfoques para esta tarea sin mucha dificultad: Una Bilinear con ResNet-50 y VGG-16, una red siamesa con ResNet-50 y otra red siamesa con VGG-16.

Un detalle importante: para hacer las redes siamesas no se cargan dos modelos idénticos en memoria (lo cual sería un desperdicio enorme de recursos). Simplemente se carga la red una vez, y para sacar las características se hace el producto externo de su salida consigo misma. Luego el backpropagation se encarga de actualizar los pesos de la red para que aprenda a sacar características que sean útiles para esta tarea.

Técnica	Accuracy % Fase 1	Accuracy % Fase 2
VGG-16 siamesa	65.94	68.11
ResNet-50 siamesa	49.36	67.73
Bilinear VGG-16 y ResNet-50	65.94	73.72

Tabla 2.2: Resultados de accuracy para diferentes redes

Tras el entrenamiento de solo la última capa (Fase 1), se obtenían unos resultados de accuracy bastante buenos para la VGG y la mixta, rondando el 65.94 %, mientras que la ResNet sola arrancó más floja. Luego, tras descongelar las CNNs y hacer el fine-tuning (Fase 2), la precisión subió en todos los frentes, destacando la combinación Bilinear (VGG + ResNet) que alcanzó casi un 74 %.

Como se puede observar, usar una sola CNN consigo misma da algo menos de accuracy que juntar dos distintas. Es lógico, ya que dos arquitecturas diferentes 'ven' cosas distintas en la imagen y se complementan mejor. Sin embargo, usar una sola red juega muy a favor de reducir el uso de memoria y potencia de cálculo, lo que lo convierte en un enfoque mucho más práctico y eficiente si vamos justos de recursos.

2.3 Ejercicio 3: Attention Visualization